AI and Machine Learning Review

Bach Pham

Source Topics Covered

- Foundations of AI, machine learning, artificial neural networks, and deep learning
- Supervised learning: regression and classification
- Linear regression, logistic regression, multinomial logistic regression
- K-nearest neighbors for classification and regression
- Decision trees for classification and regression
- Support vector machines and support vector regression
- Scaling, preprocessing, outliers, and data leakage
- Unsupervised learning: K-means, K-medoids, hierarchical clustering, DBSCAN
- Dimensionality reduction: PCA, LDA, t-SNE, UMAP
- Semi-supervised learning: self-training and label propagation
- Neural networks, CNNs, learning behavior, dropout, early stopping, augmentation, regularization
- Transfer learning, VGG16, VGG19, MobileNet, Keras applications, TensorFlow Hub

1. Big Picture of AI, ML, ANN, and Deep Learning

1.1 AI, ML, ANN, and DL Hierarchy

The hierarchy from broadest to most specialized is:

```
Artificial Intelligence
└── Machine Learning
    └── Artificial Neural Networks
        └── Deep Learning
```

Artificial Intelligence (AI) means making machines imitate or simulate human intelligence, such as reasoning, learning, decision-making, and problem-solving.

Machine Learning (ML) is a method inside AI where systems learn from data instead of being programmed with every rule manually.

Artificial Neural Networks (ANNs) are ML systems inspired by neurons in the brain. They use connected nodes, weights, biases, and activation functions to learn patterns.

Deep Learning (DL) uses large neural networks with many layers. It is powerful for raw high-dimensional data like images, audio, text, and video.

1.2 What Machine Learning Really Does

Machine learning finds a function that maps inputs to outputs:

```
y = f(x)
```

- `x` means input features.
- `y` means known label or target.
- `f(x)` is the learned model.
- `y_hat` is the model prediction.

For example:

- Predict house price from house features: regression.
- Predict spam or not spam from email word counts: classification.
- Group unlabeled flowers by similarity: clustering.
- Reduce 30 features to 2 principal components for visualization: dimensionality reduction.

1.3 General ML Workflow

Almost every ML project follows this process:

1. Collect data.
2. Prepare and clean data.
3. Choose the learning algorithm.
4. Split data into train, validation, and test sets when appropriate.
5. Train the model on training data.
6. Tune or compare models using validation data.
7. Evaluate final performance on test data.
8. Make predictions on new unseen data.

Important: the test set should represent unseen data. Do not use it to train the model, fit scalers, choose hyperparameters, or oversample.

1.4 Main Types of Machine Learning

Supervised learning uses labeled data:

```
features + label -> train model -> predict labels for new features
```

Examples:

- Regression: predict numeric value.
- Classification: predict class/category.

Unsupervised learning uses unlabeled data:

```
features only -> discover structure
```

Examples:

- Clustering.
- Dimensionality reduction.
- Association.
- Anomaly detection.

Semi-supervised learning uses a small labeled dataset plus a larger unlabeled dataset.

Reinforcement learning trains an agent through actions, environment feedback, and rewards.

Deep learning uses multi-layer neural networks, often for images, text, audio, and complex nonlinear patterns.

2. Core Vocabulary to Remember

2.1 Features, Labels, Targets, and Predictions

Feature: an input variable used by the model. Example: age, fare, sepal length, pixel intensity.

Label/target: the correct output during training. Example: survived/not survived, iris species, house price.

Prediction: output produced by the trained model.

Training data: used to fit model parameters.

Validation data: used to monitor learning and tune decisions.

Test data: used only for final evaluation.

Inference: using the trained model on new data.

2.2 Model Parameters vs Hyperparameters

Parameters are learned from data.

Examples:

- Linear regression slope and intercept
- Logistic regression coefficients
- Neural network weights and biases
- SVM support vectors and boundary parameters

- Decision tree split thresholds

Hyperparameters are chosen before or around training.

Examples:

- `k` in KNN
- `max_depth` in decision trees
- `C`, `kernel`, and `gamma` in SVM
- `epsilon` in SVR
- `n_clusters` in K-means
- `eps` and `min_samples` in DBSCAN
- `n_components` in PCA/LDA/t-SNE/UMAP
- `epochs`, `batch_size`, learning rate, dropout rate in neural networks

2.3 Parametric vs Non-Parametric

Parametric models assume a fixed form. Linear regression and logistic regression assume a linear combination of features before prediction.

Non-parametric models make fewer assumptions and can learn more flexible shapes. KNN and decision trees are common examples.

3. Data Preparation and Scaling

Good preprocessing often matters as much as the model.

3.1 Missing Values

Common options:

- Fill numeric missing values with mean or median.
- Fill categorical missing values with mode/most common value.
- Drop a column if too much data is missing and it is not useful.
- Drop rows only when missingness is small and safe.

Example from Titanic logistic regression:

- Fill missing `Age` with median age.
- Drop `Cabin` because most values are missing.
- Fill missing `Embarked` with the most common port.

3.2 Encoding Categorical Features

Models need numeric inputs. Categorical columns like `Sex` and `Embarked` must be encoded.

Common approaches:

- Label encoding: maps categories to integers.
- One-hot encoding: creates binary columns for each category.

For nominal categories without order, one-hot encoding is usually safer. Label encoding can accidentally imply order.

3.3 Feature Engineering

Feature engineering creates useful new inputs. Example from Titanic:

```
TravelAlone = 1 if SibSp + Parch == 0
TravelAlone = 0 otherwise
```

This turns family counts into a simpler feature.

3.4 Data Leakage Rules

Data leakage happens when information from the test set or future data influences training.

Remember these rules:

- Split into train/test before scaling.
- Fit the scaler only on training data.
- Transform test data using the scaler fitted on training data.
- Apply SMOTE only on training data, not before splitting.
- Apply PCA fit only on training data when building supervised models.
- Do not use test accuracy to repeatedly choose hyperparameters.

Correct pattern:

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2)

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
```

3.5 Outliers

An outlier is a data point that is far away from most observations.

Example:

```
Attendance = [75, 75, 80, 85, 85, 90, 90, 95, 30]
```

The value `30` is a likely outlier.

Types:

- Global outlier: unusual compared with the whole dataset.
- Contextual outlier: unusual in a specific context.
- Collective outlier: a group of points is unusual together.

Detection methods:

- IQR method
- Standard deviation method
- Z-score method

IQR formulas:

```
IQR = Q3 - Q1
Lower bound = Q1 - 1.5 * IQR
Upper bound = Q3 + 1.5 * IQR
```

Values outside the bounds may be outliers.

3.6 StandardScaler

StandardScaler converts values to z-scores:

```
z = (x - mean) / standard_deviation
```

Use it when:

- Data is roughly normally distributed.
- Mean and standard deviation are meaningful.
- You use SVM, KNN, logistic regression, PCA, neural networks, or similar methods.

It produces features with mean approximately 0 and standard deviation approximately 1.

3.7 MinMaxScaler

MinMaxScaler rescales values to a fixed range, often 0 to 1:

```
x_scaled = (x - x_min) / (x_max - x_min)
```

Use it when:

- You want all features between 0 and 1.
- You are working with neural networks or image pixels.
- You want a bounded scale.

Warning: it is sensitive to outliers because min and max can be extreme.

3.8 RobustScaler

RobustScaler uses median and IQR:

```
x_robust = (x - median) / IQR
```

Use it when:

- The data has outliers.
- Median and quartiles are more reliable than mean and standard deviation.

3.9 Scaling Guidelines

Scale for:

- KNN
- SVM/SVR
- Logistic regression
- PCA
- LDA usually benefits from scaling
- t-SNE/UMAP usually benefit from scaling or good feature preprocessing
- Neural networks
- CNN image pixels, usually by dividing by 255

Usually no scaling needed for:

- Decision trees
- Random forest style tree models
- Rule-based tree splitting

4. Model Evaluation

4.1 Regression Metrics

Regression predicts numeric values.

Mean Squared Error (MSE)

```
MSE = (1/n) * sum((y_i - y_hat_i)^2)
```

MSE squares errors, so large mistakes are punished heavily.

Mean Absolute Error (MAE)

```
MAE = (1/n) * sum(abs(y_i - y_hat_i))
```

MAE is easier to interpret because it stays in the target unit.

Root Mean Squared Error (RMSE)

```
RMSE = sqrt(MSE)
```

RMSE is also in the target unit, but it penalizes big errors more than MAE.

R-squared

R-squared measures how much variation in the target is explained by the model.

- `R^2 = 1`: perfect.
- `R^2 = 0`: no better than predicting the mean.
- `R^2 < 0`: worse than predicting the mean.

Adjusted R-squared

Adjusted R-squared penalizes adding unnecessary features. It is useful in multiple regression because ordinary R-squared can increase just because more variables were added.

4.2 Classification Metrics

Classification predicts classes.

Accuracy

```
accuracy = correct_predictions / total_predictions
```

Good when classes are balanced. Misleading when classes are imbalanced.

Confusion Matrix

For binary classification:

Actual / Predicted	Predicted 0	Predicted 1
Actual 0	True Negative	False Positive
Actual 1	False Negative	True Positive

For multiclass classification, each class gets its own row and column.

Precision

```
precision = TP / (TP + FP)
```

Precision answers: of all predicted positives, how many were actually positive?

Use precision when false positives are costly.

Recall

```
recall = TP / (TP + FN)
```

Recall answers: of all actual positives, how many did the model catch?

Use recall when false negatives are costly.

F1-score

```
F1 = 2 * (precision * recall) / (precision + recall)
```

F1 balances precision and recall.

Log Loss / Cross Entropy

Log loss evaluates predicted probabilities. A confident wrong prediction gets heavily penalized.

Logistic regression and neural networks often optimize cross entropy losses.

4.3 Clustering Metrics and Tools

Clustering has no labels in the usual setting, so evaluation is different.

Inertia: total distance between each point and its cluster center. Lower means more compact clusters. Used in K-means and K-medoids.

Elbow method: try multiple k values, plot inertia, and look for the point where improvement slows.

Silhouette score: measures how well a point fits its own cluster compared with other clusters. Higher is better.

Dendrogram: tree visualization for hierarchical clustering; cut at a height to choose cluster count.

Noise count: in DBSCAN, points labeled -1 are noise/outliers.

4.4 Deep Learning Curves

Watch:

- Training loss
- Validation loss
- Training accuracy
- Validation accuracy
- Test loss and test accuracy

Common patterns:

Pattern	Meaning
Train accuracy high, validation accuracy low	Overfitting
Train loss decreases, validation loss increases	Overfitting
Train and validation accuracy both low	Underfitting
Train and validation improve together	Healthy learning
Validation stops improving	Consider early stopping

5. Supervised Learning: Regression

5.1 Linear Regression

Linear regression predicts a continuous numeric value.

Simple formula:

```
y = mx + b
```

General hypothesis:

```
y_hat = beta_0 + beta_1*x
```

For multiple features:

```
y_hat = beta_0 + beta_1*x_1 + beta_2*x_2 + ... + beta_n*x_n
```

Important ideas:

- The model learns slope(s) and intercept.
- It finds the best-fit line or hyperplane.
- It minimizes prediction errors.
- It is interpretable.
- It works best when the relationship is approximately linear.

Typical use cases:

- Salary prediction from years of experience.
- House price prediction.
- Forecasting and trend modeling.

Scaling:

- Ordinary linear regression does not strictly require scaling for predictions.
- Scaling can still help with regularized regression or optimization-based workflows.

Strengths:

- Simple and explainable.
- Fast.
- Good baseline for regression.

Weaknesses:

- Cannot capture complex nonlinear patterns unless features are engineered.
- Sensitive to outliers.
- Assumes a mostly linear relationship.

5.2 KNN Regression

KNN regression predicts a numeric value by averaging the targets of the k nearest training points.

Core prediction:

```
prediction = mean(target values of k nearest neighbors)
```

Important ideas:

- Non-parametric.
- Lazy learner.
- Stores training data.
- No real training phase beyond storing data.
- Prediction can be expensive because distances to many training points are computed.

Scaling:

- Required or strongly recommended.
- Distance-based models are affected by feature scale.

k effect:

- Small k: flexible, sensitive to noise, can overfit.
- Large k: smoother, more stable, can underfit.

Use cases:

- Local patterns.
- Smaller datasets.
- Regression where similar inputs should have similar outputs.

Weaknesses:

- Slower prediction on large datasets.
- Sensitive to irrelevant features.
- Struggles in high-dimensional data.

5.3 Decision Tree Regression

Decision tree regression predicts a numeric value by splitting data into regions and predicting the average target value in each leaf.

Important terms:

- Root node: first split.
- Internal node: feature test.
- Branch: result of a test.
- Leaf node: final predicted value.
- Depth: longest path from root to leaf.

Splitting criterion often uses MSE:

```
MSE = (1/n) * sum((y_i - y_hat_i)^2)
```

For a split:

```
MSE_split = (N_left/N) * MSE_left + (N_right/N) * MSE_right
```

The best split minimizes weighted error.

Scaling:

- Not needed.
- Trees split based on thresholds and are not distance-based.

Hyperparameters:

- `max_depth`
- `min_samples_split`
- `min_samples_leaf`
- `criterion`

Overfitting:

- Deep trees can memorize training data.
- Limit `max_depth` or use pruning.

Pruning:

- Pre-pruning: stop growth early with hyperparameters like `max_depth`.
- Post-pruning: grow tree then cut back branches, such as with cost complexity pruning.

Use cases:

- Nonlinear tabular regression.
- Interpretable prediction rules.
- Mixed feature scales.

Weaknesses:

- Can overfit.
- Predictions are step-like, not smooth.
- Small data changes can change tree structure.

5.4 Support Vector Regression (SVR)

SVR is the regression version of SVM.

Main idea:

- Find a function that predicts the target while allowing an error tube.
- Points inside the `epsilon`-insensitive tube receive no penalty.
- Points outside the tube are penalized.

Important parameters:

- `kernel`: linear, RBF, polynomial, sigmoid.
- `C`: controls penalty for errors outside the tube.
- `epsilon`: tube width; larger epsilon ignores larger errors.
- `gamma`: controls RBF kernel influence.

Scaling:

- Required or strongly recommended.

Kernel behavior:

- Linear SVR: straight relationship.
- RBF SVR: nonlinear flexible curve.
- Polynomial SVR: structured curved relationship.

Example from notes:

- Student marks prediction using `time_study`.
- Linear SVR, RBF SVR, and polynomial SVR compared.
- Higher `C` made RBF SVR fit more tightly and improved RMSE/R^2 in that example.

Use cases:

- Smaller to medium datasets.
- Nonlinear regression.
- Time series or continuous prediction where margin tolerance is useful.

Weaknesses:

- Needs scaling.
- Sensitive to hyperparameters.
- Can be slow on large datasets.
- Less interpretable than linear regression or decision tree.

6. Supervised Learning: Classification

6.1 Logistic Regression

Despite the name, logistic regression is used for classification.

It starts with a linear score:

```
z = beta_0 + beta_1*x_1 + ... + beta_n*x_n
```

Then it applies the sigmoid function:

```
sigmoid(z) = 1 / (1 + e^(-z))
```

The output is a probability between 0 and 1.

Decision boundary:

```
if p >= 0.5 -> class 1
if p < 0.5 -> class 0
```

Loss:

- Logistic regression uses cross entropy/log loss.

Scaling:

- Strongly recommended.
- It uses optimization and feature weights, so scaling improves convergence and fair weighting.

Example from notes:

- Titanic survival prediction.
- Preprocessing included missing value imputation, dropping high-missing columns, encoding categories, and feature engineering.
- Accuracy about 79.48%.
- Classification report and confusion matrix used for evaluation.

Strengths:

- Simple and interpretable.
- Produces probabilities.
- Strong baseline for binary classification.

Weaknesses:

- Linear decision boundary unless features are engineered.
- Can struggle with complex nonlinear data.

6.2 Multinomial Logistic Regression

Multinomial logistic regression handles classification with three or more unordered classes.

Example:

- Dry Bean dataset with seven classes:
  - Seker
  - Barbunya
  - Bombay
  - Cali
  - Dermosan
  - Horoz

    - Sira

Important notes:

- Dry Bean has many numerical features on different scales.
- The dataset is imbalanced.
- SMOTE can create synthetic minority class samples.
- SMOTE must be applied only to the training data.
- Scaling improved accuracy in the notes from about 71% to about 91%.

Correct workflow:

1. Split data.
2. Fit scaler on training data.
3. Transform train and test.
4. Apply SMOTE to scaled training data only.
5. Train multinomial logistic regression.
6. Evaluate on untouched test data.

Example model:

```
model = LogisticRegression(multi_class="multinomial", solver="lbfgs")
```

6.3 KNN Classification

KNN classifier predicts a class by majority vote among the k nearest neighbors.

Important ideas:

- Supervised.
- Non-parametric.
- Lazy learner.
- Distance-based.
- Works for classification and regression.

Prediction process:

1. Choose k.
2. Compute distance from new point to training points.
3. Select k closest points.
4. Use majority vote.

k values:

- $k = 1$: very sensitive to noise, can overfit.
- $k = 2$: ties are common.
- $k = 3$ or odd values: often better for avoiding ties.
- Large k: smoother boundary but may underfit.

Scaling:

- Required or strongly recommended.

Distance metrics:

- Euclidean distance.
- Manhattan distance.

Strengths:

- Simple.
- Works with little assumptions.
- Easy to explain.
- Can work well with small or medium datasets.

Weaknesses:

- Prediction can be slow.
- Requires storing training data.
- Sensitive to scaling and irrelevant features.
- Not naturally ideal for categorical data.

6.4 Decision Tree Classification

Decision trees classify by recursively splitting data based on feature rules.

Key components:

- Root node.
- Internal nodes.
- Leaf nodes.
- Branches.
- Depth.
- Splitting.

Common split criteria:

Gini Impurity

For two classes:

```
Gini = 1 - p_A^2 - p_B^2
```

Lower Gini is better.

Interpretation:

- Gini = 0 means pure node.
- Higher Gini means more mixed classes.

Entropy

```
Entropy = - sum(p_i * log2(p_i))
```

Lower entropy means more purity.

Information Gain

Information gain measures how much uncertainty is reduced after a split.

Higher information gain is better.

Important tree parameters:

- `max_depth`: maximum depth.
- `min_samples_split`: minimum samples needed to split.
- `min_samples_leaf`: minimum samples in a leaf.
- `criterion`: `gini` or `entropy`.
- `max_features`: number of features considered at each split.

Scaling:

- Not needed.

Example from notes:

- Iris dataset with two features gave about 76% test accuracy.
- Iris dataset with all four features gave 100% test accuracy in the example.
- Petal width and petal length became important features.

Spam email example:

- 5172 rows, 3002 columns.
- 3000 word-count features.
- `Prediction` column: 0 = not spam, 1 = spam.
- `max_depth`=3 gave train/test accuracy around 85%.
- No-limit tree reached higher training accuracy and better test accuracy in one run but had more overfitting risk.

Strengths:

- Interpretable.
- No scaling required.
- Handles nonlinear rules.

- Can show feature importance.

Weaknesses:

- Can overfit.
- Unstable with small data changes.
- Greedy splitting may not find globally best tree.

6.5 Support Vector Machine (SVM) Classification

SVM finds the best separating hyperplane between classes.

Key terms:

- Hyperplane: decision boundary.
- Margin: distance between hyperplane and closest data points.
- Support vectors: closest points that define the margin.
- Hard margin: no misclassification allowed.
- Soft margin: allows some errors for better generalization.

Why maximize margin?

- Small margin can overfit and misclassify new points near boundary.
- Large margin usually generalizes better.

Important parameter:

```
C
```

- Low `C`: wider margin, more tolerance for misclassification.
- High `C`: narrower margin, stronger penalty for misclassification.

Scaling:

- Required or strongly recommended.

Linear SVM:

```
LinearSVC(C=1, loss="hinge")
SVC(C=3, kernel="linear")
```

RBF kernel:

- Flexible curved boundary.
- Often default for `SVC`.
- Can overfit if not tuned.

Polynomial kernel:

- Captures feature interactions.
- More expensive than linear.

Sigmoid kernel:

- Related to neural-network style activation.
- Can be unstable.

Example from notes:

- With Iris using only sepal length and sepal width, linear and RBF struggled because two features were not enough.
- With all four Iris features, both linear and RBF reached 100% accuracy in the example.
- KNN with k=9 performed better than RBF SVM on the two-feature Iris example.

Strengths:

- Good for high-dimensional spaces.
- Effective with clear margins.
- Kernels handle nonlinear boundaries.

Weaknesses:

- Needs scaling.
- Can be slow on large datasets.

- Hyperparameter tuning matters.
- Less interpretable than trees or linear models.

7. Unsupervised Learning

Unsupervised learning finds structure in unlabeled data.

Common types:

- Clustering.
- Dimensionality reduction.
- Association.
- Anomaly detection.

7.1 Clustering

Clustering groups similar data points.

Useful when:

- Labels are unknown.
- You want to discover natural groups.
- You want exploratory analysis.
- You want to segment customers, patients, images, or documents.

7.2 K-Means Clustering

K-means is a partitional clustering algorithm.

Process:

1. Choose k, number of clusters.
2. Randomly initialize k centroids.
3. Assign each point to nearest centroid.
4. Recompute each centroid as the mean of its assigned points.
5. Repeat until centroids stop changing.

Important parameters:

- `n_clusters`
- `random_state`
- `n_init`

Important output:

- `cluster_centers_`: centroids.
- `labels_`: cluster ID for each point.
- `inertia_`: total within-cluster distance.

Scaling:

- Usually needed because K-means is distance-based.

Choosing k:

- Use elbow method.
- Try values like 1 to 10.
- Pick the point where inertia stops decreasing sharply.

Strengths:

- Fast.
- Simple.
- Good for large datasets with round, well-separated clusters.

Weaknesses:

- Must choose k.
- Sensitive to outliers.
- Sensitive to initialization.
- Assumes clusters are roughly spherical.

7.3 K-Medoids

K-medoids is similar to K-means, but cluster centers are actual data points called medoids.

Main difference:

- K-means center = mathematical mean.
- K-medoids center = actual data point.

Strengths:

- More robust to outliers.
- More interpretable cluster centers.

Weaknesses:

- Usually slower than K-means.
- Inertia can be higher because medoids are not perfect averages.

Use cases:

- Smaller datasets with noise or outliers.
- When a real representative data point is useful.

7.4 Hierarchical Clustering

Hierarchical clustering builds nested clusters.

Two approaches:

- Agglomerative: bottom-up, start each point as its own cluster and merge.
- Divisive: top-down, start with all points and split.

The notes focus on agglomerative clustering.

Important parameters:

- `n_clusters`
- `metric`
- `linkage`

Common linkage methods:

- `ward`
- `complete`
- `average`
- `single`

Important rule:

- If linkage is `ward`, metric must be Euclidean.

7.5 Dendrogram

A dendrogram is a tree-like diagram showing how clusters merge.

Interpretation:

- Bottom: individual points or small groups.
- Height: distance/dissimilarity at which clusters merge.
- Large vertical gaps suggest natural cluster separations.
- Draw a horizontal line to choose number of clusters.

7.6 Silhouette Score

Silhouette score helps choose number of clusters.

- Higher is better.
- Compares how close points are to their own cluster versus other clusters.
- In the Iris example, silhouette score selected 2 clusters even though the known species count is 3. That can happen because two species overlap.

7.7 DBSCAN

DBSCAN means Density-Based Spatial Clustering of Applications with Noise.

It groups dense regions and labels isolated points as noise.

Key terms:

- `eps`: maximum distance for points to be neighbors.
- `min_samples`: minimum points needed in an epsilon-neighborhood.
- Core point: has enough neighbors.
- Border point: near a core point but not dense enough itself.
- Noise point: neither core nor border, labeled `-1`.

Important:

- DBSCAN does not require specifying number of clusters.
- It is good for arbitrary-shaped clusters.
- It can detect outliers.
- It does not naturally use train/test split in the same way supervised models do.

Parameter effects:

- Too large `eps`: clusters merge into one big cluster.
- Too small `eps`: many noise points and fragmented clusters.
- Higher `min_samples`: stricter density requirement, more noise.
- Lower `min_samples`: easier to form clusters, possibly too many clusters.

Example from notes:

- Iris with `eps=0.5`, `min_samples=5` found 1 cluster and 2 noise points.
- Iris with `eps=0.3` found 3 clusters and 12 noise points.
- Changing `min_samples` changed cluster count and noise count.

8. Dimensionality Reduction

Dimensionality reduction represents high-dimensional data in fewer dimensions while preserving important information.

Reasons to use it:

- Reduce overfitting.
- Reduce computation.
- Remove redundancy.
- Improve visualization.
- Help models learn from correlated features.

Important terms:

- Dimension space: number of features.
- Projection: mapping high-dimensional data to lower-dimensional space.
- Orthogonality: components are decorrelated or independent in direction.

8.1 PCA: Principal Component Analysis

PCA is an unsupervised linear dimensionality reduction method.

Main idea:

- Find directions of maximum variance.
- Project data onto those directions.
- Principal components are new variables.
- Components are ordered by explained variance.

Important:

- PCA requires scaling before use.
- PCA combines correlated features.
- It can reduce interpretability because components are not original features.

Example from notes:

- Breast cancer dataset has 569 samples and 30 features.

- Many features are highly correlated, such as mean radius and mean perimeter.
- PCA reduced 30 features to fewer components.
- First two components explained about 44% and 19% variance.
- `n_components=0.95` kept enough components to preserve about 95% variance and selected 10 components.

Common code:

```
scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
```

For supervised modeling:

```
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)
```

Do not fit PCA on test data.

8.2 LDA: Linear Discriminant Analysis

LDA is a supervised linear dimensionality reduction method.

Main idea:

- Find projections that maximize class separability.
- Uses labels.
- Good for classification.

Difference from PCA:

- PCA ignores labels and preserves variance.
- LDA uses labels and preserves class separation.

Maximum number of LDA components:

```
n_components <= min(n_classes - 1, n_features)
```

Examples:

- Breast cancer: 2 classes, 30 features.
  - Max components = min(1, 30) = 1.
- Iris: 3 classes, 4 features.
  - Max components = min(2, 4) = 2.

The notes mention Iris as `min(4 - 1, 4)`, but Iris has 3 classes, so the correct class-based limit is `min(3 - 1, 4) = 2`.

Scaling:

- Usually recommended.

Example:

- Breast cancer reduced from `(455, 30)` training shape to `(455, 1)`.
- Logistic regression on LDA projection reached about 94.7% accuracy.

8.3 t-SNE

t-SNE is an unsupervised nonlinear dimensionality reduction method, mainly for visualization.

Main idea:

- Compute similarity probabilities in high-dimensional space.
- Map points into 2D or 3D.
- Preserve local neighbor structure.
- Use gradient descent to arrange points.

Important parameter:

- `perplexity`: roughly controls how many neighbors each point considers.

Use cases:

- Visualizing high-dimensional data.
- Visualizing CNN feature embeddings.
- Seeing class clusters after feature extraction.

Warnings:

- Mostly for visualization, not usually a preprocessing step for supervised model training.
- Can be slow on large datasets.
- Global distances may be misleading.
- Results depend on random state and parameters.

Example from notes:

- Breast cancer data reduced to 2D.
- CNN features from MNIST: features of shape `(2000, 64)` reduced to `(2000, 2)`.
- Imagenette/MobileNetV2 CNN features: `(5000, 1280)` reduced to `(5000, 2)`.

8.4 UMAP

UMAP means Uniform Manifold Approximation and Projection.

It is a nonlinear dimensionality reduction method that preserves local and some global structure.

Important parameters:

- `n_components`: output dimension.
- `n_neighbors`: local vs global balance.
- `min_dist`: how tightly points can pack.
- `random_state`: reproducibility.

Parameter effects:

- Small `n_neighbors`: focuses on local structure.
- Large `n_neighbors`: considers broader/global structure.
- Small `min_dist`: tighter clusters.
- Large `min_dist`: more spread-out embedding.

UMAP vs t-SNE:

- UMAP is usually faster.
- UMAP can scale better to larger datasets.
- UMAP may preserve more global structure.
- Both are useful for visualization.

9. Semi-Supervised Learning

Semi-supervised learning uses:

- Small labeled dataset.
- Large unlabeled dataset.

It helps when labels are expensive but unlabeled data is available.

9.1 Self-Training

Self-training process:

1. Train a supervised base model on labeled data.
2. Predict labels for unlabeled data.
3. Keep only confident pseudo-labels.
4. Add pseudo-labeled data to training set.
5. Retrain.

Example:

- Breast cancer dataset.

- Keep only 10% labels.
- Mark the rest as `-1`.
- Use logistic regression as base model.
- Use `SelfTrainingClassifier`.
- Threshold `0.8` means only add unlabeled samples if confidence is at least 80%.

Important code idea:

```
y_train_semi = np.copy(y_train)
y_train_semi[~mask] = -1

base_model = LogisticRegression(max_iter=1000)
model = SelfTrainingClassifier(estimator=base_model, threshold=0.8)
model.fit(X_train_scaled, y_train_semi)
```

In the notes, self-training reached about 98.25% accuracy.

9.2 Label Propagation

Label propagation builds a graph:

- Each data point is a node.
- Edges connect similar points.
- Labels spread from labeled nodes to unlabeled nodes.

Example:

```
lp = LabelPropagation(kernel="rbf")
lp.fit(X_train_scaled, y_train_semi)
```

Strengths:

- Uses structure of all data points.
- Can work well when similar points share labels.

Weaknesses:

- Can perform badly with noisy data.
- Sensitive to similarity graph quality.

In the notes, label propagation reached about 92.1% accuracy.

10. Neural Networks

10.1 What Is a Neural Network?

A neural network is made of connected neurons/nodes arranged in layers.

Basic layers:

- Input layer: receives features.
- Hidden layers: learn intermediate patterns.
- Output layer: produces prediction.

Each neuron computes:

```
z = sum(w_i*x_i) + b
```

Then applies an activation function:

```
output = activation(z)
```

Weights and biases are learned during training.

10.2 Bias

Bias is like the intercept in linear regression:

```
y = ax + b
```

It lets the model shift the function up or down and fit patterns that do not pass through the origin.

10.3 Activation Functions

Sigmoid

```
sigmoid(x) = 1 / (1 + e^(-x))
```

Range: 0 to 1.

Use:

- Binary classification output layer.

Tanh

```
tanh(x) = (e^x - e^(-x)) / (e^x + e^(-x))
```

Range: -1 to 1.

Use:

- Hidden layers in some networks.

ReLU

```
f(x) = max(0, x)
```

Use:

- Hidden layers.
- CNNs.
- Helps introduce nonlinearity and train efficiently.

Softmax

```
softmax(z_i) = e^(z_i) / sum(e^(z_j))
```

Use:

- Multiclass classification output layer.

Softmax outputs probabilities that sum to 1.

10.4 Neural Network Types

Feedforward neural networks: information moves from input to output with no loops.

CNNs: designed for images and spatial data.

RNNs: designed for sequential data.

LSTMs: special RNNs that handle long-term dependencies using gates.

Transformers: modern deep learning architecture used heavily in language, vision, and multimodal tasks.

10.5 Basic Keras Workflow

1. Import libraries.
2. Prepare features and labels.

3. Build `Sequential` model.
4. Add layers.
5. Compile model.
6. Fit/train model.
7. Evaluate model.
8. Plot loss and accuracy.

Example binary classification network:

```
model = keras.Sequential([
    keras.layers.Dense(4, activation="relu", input_dim=2),
    keras.layers.Dense(1, activation="sigmoid")
])

model.compile(
    optimizer="adam",
    loss="binary_crossentropy",
    metrics=["accuracy"]
)

history = model.fit(X, y, epochs=30, batch_size=32)
```

Important terms:

- `epoch`: one full pass through training data.
- `batch_size`: number of samples processed before weight update.
- `optimizer`: method that updates weights.
- `loss`: function the model minimizes.
- `metrics`: values tracked for evaluation.

11. CNNs and Image Classification

CNNs are designed for image data because they preserve spatial relationships between pixels and learn visual features.

11.1 MNIST Dataset

MNIST:

- 70,000 handwritten digit images.
- 60,000 training images.
- 10,000 test images.
- Each image is 28 x 28 grayscale.
- Pixel values range from 0 to 255.
- Labels are digits 0 through 9.

11.2 Image Preprocessing

For dense neural network:

```
X_train = X_train / 255
X_test = X_test / 255
```

For CNN:

```
X_train = X_train.reshape(X_train.shape[0], 28, 28, 1)
X_test = X_test.reshape(X_test.shape[0], 28, 28, 1)

X_train = X_train.astype("float32") / 255
X_test = X_test.astype("float32") / 255
```

Shape meaning:

```
(60000, 28, 28, 1)
```

- 60000 images.

- height 28.
- width 28.
- 1 channel for grayscale.

RGB images use 3 channels.

11.3 Dense Network for MNIST

Example:

```
model = keras.Sequential()
model.add(Flatten(input_shape=(28, 28)))
model.add(Dense(128, activation="relu"))
model.add(Dense(32, activation="relu"))
model.add(Dense(10, activation="softmax"))
```

Why flatten?

- Converts 28 x 28 image to 784-element vector.
- Dense layers need 1D input.

Important correction:

- Flattening does not preserve spatial relationships the way CNN convolution does.
- It simply makes the image usable by dense layers.

Output layer:

- 10 neurons for digits 0-9.
- Softmax for multiclass classification.

Loss:

```
loss="sparse_categorical_crossentropy"
```

Use this when labels are integer class IDs, not one-hot vectors.

11.4 CNN Architecture

Common CNN pipeline:

```
Input -> Convolution + ReLU -> Pooling -> Convolution + ReLU -> Pooling -> Flatten -> Dense -> Output
```

Layers:

- Input layer: image data.
- Convolution layer: feature extraction.
- ReLU: nonlinearity.
- Pooling layer: downsampling.
- Flatten: turns feature maps into 1D vector.
- Fully connected layer: combines learned features for classification.
- Softmax/logistic output: class probabilities.

11.5 Convolution

Convolution uses a filter/kernel that slides over an image.

Example:

- Input image: 5 x 5.
- Kernel: 3 x 3.
- Output: feature map.

The convolution operation multiplies corresponding values and sums them.

Important Conv2D parameters:

- `filters`: number of filters learned.
- `kernel_size`: filter size, such as `(3, 3)`.

- `strides`: how far the filter moves.
- `padding`: `"valid"` or `"same"`.
- `activation`: often `"relu"`.
- `input_shape`: image height, width, channels.

Example:

```
Conv2D(
    32,
    kernel_size=(3, 3),
    strides=(1, 1),
    padding="valid",
    activation="relu",
    input_shape=(28, 28, 1)
)
```

11.6 Filters

Filters learn visual patterns like:

- Edges.
- Corners.
- Curves.
- Textures.
- Object parts.

Early layers learn simple features. Deeper layers learn more complex features.

Common filter counts:

- Early layers: 32, 64, 128.
- Deeper layers: 256, 512, 1024.

11.7 Kernel Size

Common kernel sizes:

- 1 x 1.
- 3 x 3.
- 5 x 5.
- 7 x 7.

VGG architectures famously use 3 x 3 filters repeatedly.

11.8 Padding

`padding="valid"`:

- No padding.
- Output spatial size shrinks.

`padding="same"`:

- Adds padding so output size stays same when stride is 1.

For input 28 x 28, kernel 3 x 3, stride 1, valid padding:

```
output = 28 - 3 + 1 = 26
```

So output is 26 x 26.

11.9 Stride

Stride is how many pixels the filter moves each step.

- Stride 1: moves one pixel at a time.
- Stride 2: moves two pixels at a time and reduces output size.

11.10 Pooling

Pooling reduces spatial dimensions and keeps important information.

Types:

- Max pooling: take maximum value.
- Average pooling: take average.
- Sum pooling: take sum.

Example max pooling:

Input:

```
1 1 2 4
5 6 7 8
3 2 1 0
1 2 3 4
```

2 x 2 max pooling with stride 2:

```
6 8
3 4
```

11.11 CNN Example Model

```
model = Sequential()

model.add(Conv2D(
    32,
    kernel_size=(3, 3),
    strides=(1, 1),
    padding="valid",
    activation="relu",
    input_shape=(28, 28, 1)
))

model.add(MaxPool2D(pool_size=(2, 2)))
model.add(Flatten())
model.add(Dense(100, activation="relu"))
model.add(Dense(10, activation="softmax"))
```

Compile:

```
model.compile(
    loss="sparse_categorical_crossentropy",
    metrics=["accuracy"],
    optimizer="adam"
)
```

Train:

```
model.fit(
    X_train,
    y_train,
    batch_size=128,
    epochs=10,
    validation_split=0.2
)
```

Evaluate:

```
test_loss, test_accuracy = model.evaluate(X_test, y_test)
```

11.12 CNN Parameter Calculation

For Conv2D:

```
parameters = ((kernel_height * kernel_width * input_channels) + bias) * filters
```

Example:

```
input shape = 28 x 28 x 1
filters = 32
kernel = 3 x 3
parameters = ((3*3*1) + 1) * 32 = 320
```

Max pooling has no learnable parameters.

Flatten has no learnable parameters.

Dense layer:

```
parameters = (previous_units * current_units) + current_units
```

Example:

```
Flatten output = 13 * 13 * 32 = 5408
Dense(100) params = (5408 * 100) + 100 = 540900
Dense(10) params = (100 * 10) + 10 = 1010
```

12. Learning Behavior: Overfitting and Underfitting

12.1 Overfitting

Overfitting means the model learns training data too well, including noise, and performs poorly on unseen data.

Signs:

- Training accuracy very high.
- Validation/test accuracy much lower.
- Training loss decreases.
- Validation loss increases.

Causes:

- Training dataset too small.
- Model too complex.
- Too much noise.
- Irrelevant features.
- Too many epochs.

Fixes:

- Add dropout.
- Use early stopping.
- Use data augmentation.
- Use regularization.
- Reduce model complexity.
- Get more data.
- Improve preprocessing.

12.2 Dropout

Dropout randomly turns off neurons during training.

Example:

```
keras.layers.Dropout(rate=0.4)
```

Meaning:

- 40% of selected neurons are dropped during each training step.
- Dropout is not active during normal prediction after training.

Why it helps:

- Prevents neurons from relying too much on each other.
- Acts like an ensemble of many smaller networks.
- Improves generalization.

Common CNN dropout rates:

- Around 0.4 or 0.5 in the notes.

12.3 Early Stopping

Early stopping stops training when validation performance stops improving.

Example:

```
callbacks = [
    EarlyStopping(
        monitor="val_accuracy",
        patience=5,
        restore_best_weights=True
    )
]
```

Use in training:

```
history = model.fit(
    X_train,
    y_train,
    epochs=25,
    validation_split=0.2,
    callbacks=callbacks
)
```

Important terms:

- `monitor`: metric to watch.
- `patience`: how many epochs without improvement before stopping.
- `restore_best_weights=True`: return to best model weights.

12.4 Data Augmentation

Data augmentation creates modified training examples, especially useful for images.

Examples:

- Random rotations.
- Random crop.
- Random flips.
- Random brightness.
- Random zoom.

Keras examples:

```
tf.keras.layers.RandomFlip("horizontal")
tf.keras.layers.RandomRotation(0.1)
tf.keras.layers.RandomZoom(0.1)
```

Important:

- Use augmentation during preprocessing/training.
- Too much augmentation can cause underfitting.

12.5 Regularization

Regularization discourages overly complex models.

L1 regularization:

- Adds penalty based on absolute weights.
- Can force some weights to zero.
- Useful for feature selection.

L2 regularization:

- Adds penalty based on squared weights.
- Discourages very large weights.
- Encourages smoother models.

12.6 Underfitting

Underfitting means the model is too simple to learn the pattern.

Signs:

- Training accuracy low.
- Validation accuracy low.
- Training and validation errors both high.

Fixes:

- Use larger model.
- Add layers, neurons, or filters.
- Train longer.
- Reduce too-strong regularization.
- Remove excessive dropout.
- Remove excessive augmentation.
- Add better features.

13. Transfer Learning

Transfer learning uses a model trained on a large dataset as a starting point for a new task.

It is especially useful when:

- Your dataset is small.
- Your task is similar to the pretrained model's task.
- You want faster training.
- You want better feature extraction.

13.1 Common CNN Architectures

Examples:

- LeNet
- AlexNet
- VGGNet
- GoogLeNet
- ResNet
- ZFNet
- MobileNet
- Inception
- Xception

13.2 VGGNet

VGGNet was developed by Oxford's Visual Geometry Group.

Important ideas:

- Deep CNN architecture.
- Uses many 3 x 3 convolution filters.
- Uses ReLU.
- Uses max pooling.

- Final layers are fully connected and softmax.

VGG variants:

- VGG11
- VGG13
- VGG16
- VGG19

The number refers to layers with trainable weights.

13.3 VGG16

VGG16:

- 13 convolutional layers.
- 5 max pooling layers.
- 3 dense layers.
- Input size usually 224 x 224 x 3.
- Uses 3 x 3 filters, stride 1, same padding.
- Max pooling is 2 x 2 with stride 2.
- Final dense output has 1000 ImageNet classes when `include_top=True`.

13.4 VGG19

VGG19:

- 16 convolutional layers.
- 5 max pooling layers.
- 3 dense layers.
- 19 trainable layers.
- Input size usually 224 x 224 x 3.
- Very large, about 143.7M parameters in full ImageNet configuration.

VGG19 blocks:

- Block 1: 64 filters.
- Block 2: 128 filters.
- Block 3: 256 filters.
- Block 4: 512 filters.
- Block 5: 512 filters.

Then:

- Flatten.
- Dense layers.
- Softmax output.

13.5 Keras Applications

Keras provides pretrained models:

- VGG16
- VGG19
- ResNet50
- ResNet101
- InceptionV3
- Xception
- MobileNet
- MobileNetV2
- and more

Example:

```
from tensorflow.keras.applications import VGG19

model = VGG19(weights="imagenet")
```

13.6 Important VGG19 Parameters

```
keras.applications.VGG19(
    include_top=True,
    weights="imagenet",
    input_shape=None,
    pooling=None,
    classes=1000,
    classifier_activation="softmax"
)
```

Important parameters:

- `include_top=True`: keep original classifier head.
- `include_top=False`: remove final dense classification layers.
- `weights="imagenet"`: use pretrained ImageNet weights.
- `input_shape`: custom input shape, usually used when `include_top=False`.
- `pooling`: optional pooling for feature extraction.
- `classes`: number of output classes if training from scratch with top included.

13.7 Image Prediction with Pretrained VGG

Workflow:

1. Load image.
2. Resize to 224 x 224.
3. Convert to NumPy array.
4. Reshape to include batch dimension.
5. Preprocess with model-specific `preprocess_input`.
6. Predict probabilities.
7. Decode predictions.

Example:

```
image = load_img("Coffee-Mug.jpg", target_size=(224, 224))
image = img_to_array(image)
image = image.reshape((1, image.shape[0], image.shape[1], image.shape[2]))
image = preprocess_input(image)

prediction = model.predict(image)
label = decode_predictions(prediction)
label = label[0][0]
```

13.8 Transfer Learning Process

Steps:

1. Select a pretrained model.
2. Create a base model with `include_top=False`.
3. Freeze base model layers.
4. Add new trainable layers.
5. Train the new head on your dataset.
6. Optionally fine-tune some upper layers later.

Example:

```
base_model = VGG16(
    include_top=False,
    weights="imagenet",
    input_shape=(224, 224, 3)
)

base_model.trainable = False
```

Why freeze layers?

- Preserve pretrained knowledge.
- Reduce computation.

- Prevent overfitting on small datasets.
- Avoid destroying useful pretrained weights.

After freezing, add custom layers:

```
model = keras.Sequential([
    base_model,
    keras.layers.GlobalAveragePooling2D(),
    keras.layers.Dense(128, activation="relu"),
    keras.layers.Dense(num_classes, activation="softmax")
])
```

For binary classification, final layer can be:

```
keras.layers.Dense(1, activation="sigmoid")
```

13.9 MobileNet

MobileNet is much smaller and faster than VGG.

Use when:

- You need lightweight inference.
- You have limited compute.
- You want transfer learning with fewer parameters.

13.10 TensorFlow Hub

TensorFlow Hub provides reusable pretrained models and embeddings.

Example from notes:

```
import tensorflow_hub as hub

model = hub.KerasLayer("https://tfhub.dev/google/nnlm-en-dim128/2")
embeddings = model(["The rain in Spain.", "falls", "mainly", "In the plain!"])
```

14. Practical Workflows to Memorize

14.1 Tabular Classification Workflow

Use for Titanic, Dry Bean, spam email, Iris, etc.

1. Load data.
2. Inspect shape, columns, missing values.
3. Separate X and y.
4. Encode categorical variables.
5. Split train/test.
6. Scale if model requires it.
7. Apply SMOTE only to training data if classes are imbalanced.
8. Train model.
9. Predict test set.
10. Evaluate with accuracy, confusion matrix, classification report.

14.2 Tabular Regression Workflow

Use for salary, Boston housing, diabetes, auto MPG, student marks.

1. Load data.
2. Select features and target.
3. Split train/test.
4. Scale if using KNN/SVR/neural network.
5. Train model.
6. Predict test set.

7. Evaluate with MAE, MSE, RMSE, R^2.
8. Compare train vs test performance to detect overfitting.

14.3 Clustering Workflow

1. Load features only.
2. Scale if distance-based.
3. Choose clustering algorithm.
4. Choose initial hyperparameters.
5. Fit model.
6. Plot clusters if 2D or after dimensionality reduction.
7. Use inertia/elbow, silhouette score, dendrogram, or noise count.
8. Interpret clusters with domain knowledge.

14.4 PCA + Supervised Model Workflow

1. Split train/test.
2. Fit scaler on training data.
3. Transform train/test.
4. Fit PCA on scaled training data.
5. Transform scaled train/test with PCA.
6. Train classifier/regressor.
7. Evaluate on transformed test data.

14.5 CNN Image Classification Workflow

1. Load images and labels.
2. Split train/test or use provided split.
3. Normalize pixels to 0-1.
4. Reshape to include channels.
5. Build CNN.
6. Compile with correct loss.
7. Train with validation split.
8. Plot train vs validation loss/accuracy.
9. Evaluate on test set.
10. Use classification report for class-level performance.

14.6 Transfer Learning Workflow

1. Pick pretrained model.
2. Use correct input size and preprocessing.
3. Load with `include_top=False`.
4. Freeze base model.
5. Add custom classifier head.
6. Train new layers.
7. Evaluate.
8. Optionally unfreeze top layers and fine-tune with small learning rate.

15. Exam Traps and Memory Rules

- Logistic regression is for classification, not regression.
- Multinomial logistic regression is for three or more unordered classes.
- Ordinal logistic regression is for ordered categories.
- Linear regression predicts continuous values.
- Classification predicts categories.
- Regression predicts numbers.
- KNN and SVM need scaling.
- Decision trees do not need scaling.
- PCA needs scaling.
- PCA is unsupervised.
- LDA is supervised.
- t-SNE and UMAP are mainly visualization methods.
- DBSCAN does not require choosing number of clusters.
- K-means requires choosing number of clusters.
- K-means uses centroids; K-medoids uses actual data points.

- K-means is sensitive to outliers; K-medoids is more robust.
- DBSCAN labels noise as `-1`.
- Fit scalers only on training data.
- SMOTE only on training data.
- Test data must remain untouched until final evaluation.
- High training score and low test score means overfitting.
- Low training score and low test score means underfitting.
- Increasing tree depth can increase overfitting.
- Increasing KNN `k` smooths predictions.
- Increasing SVM `C` usually penalizes errors more strongly and can create a tighter fit.
- Increasing SVR `epsilon` makes the model tolerate larger errors inside the tube.
- Softmax is for multiclass output.
- Sigmoid is for binary probability output.
- ReLU is common in hidden layers.
- CNNs are preferred over SVMs for raw image pixels because CNNs learn visual features automatically.
- Transfer learning is valuable when your image dataset is small.
- Freeze pretrained layers first; train the new classifier head.

16. Complete Theory Deep Dive

This section gathers the theory across all topics so the document is useful for essay questions, short-answer questions, and "explain why" questions, not only coding questions.

16.1 Why Machine Learning Matters

Traditional programming solves problems by writing explicit rules:

```
human writes rules + input data -> output
```

Machine learning changes the process:

```
input data + correct outputs -> algorithm learns rules/model
```

This is useful when:

- The rules are too complex to manually write.
- There are hidden patterns in large datasets.
- The system should improve with experience.
- We need prediction, classification, grouping, detection, or automation at scale.

Examples:

- Detecting fraud from transaction patterns.
- Diagnosing disease from medical images.
- Predicting house prices.
- Classifying spam emails.
- Grouping customers by behavior.
- Recognizing handwritten digits.

Advantages:

- Can process huge data efficiently.
- Can discover hidden relationships.
- Can automate routine tasks.
- Can scale as data grows.
- Can improve security, quality control, recommendation, diagnosis, and forecasting.

Disadvantages:

- Needs high-quality data.
- Can be biased if data is biased.
- Can require expensive computation.
- Can be hard to interpret.
- Needs maintenance because data patterns can change over time.
- Can raise privacy and ethical concerns.

16.2 Evolution Theory: Why the Field Developed This Way

Important milestones:

- 1943: McCulloch and Pitts proposed a mathematical neuron model.
- 1957/1958: Rosenblatt introduced the perceptron for pattern recognition.
- 1960s: Decision trees and linear regression became important classical ML tools.
- 1970s: Rule-based expert systems were popular.
- 1980s: Backpropagation made multilayer neural networks more practical.
- 1990s-2000s: CNNs, LSTMs, and larger datasets became important.
- 2000s: Big data increased the need for scalable ML.
- 2010s-present: Deep learning expanded through GPUs, large datasets, better optimizers, CNNs, transformers, and generative AI.

Theory lesson:

- Early AI relied heavily on human-written rules.
- Classical ML learns patterns from structured features.
- Deep learning learns features directly from raw data.
- Transfer learning reuses features learned from massive datasets.

16.3 Training, Validation, Testing, and Generalization Theory

The goal is not just to perform well on training data. The real goal is **generalization**, meaning good performance on unseen data.

Data splits:

- Training set: used to learn parameters.
- Validation set: used to tune choices and monitor overfitting.
- Test set: used for final unbiased evaluation.

The model should never "peek" at the test set during training or preprocessing decisions.

Generalization depends on:

- Amount of data.
- Quality of features.
- Noise level.
- Model complexity.
- Regularization.
- Hyperparameter choices.
- Whether train and test data come from similar distributions.

16.4 Bias-Variance Theory

Bias and variance explain underfitting and overfitting.

High bias:

- Model is too simple.
- It misses the true pattern.
- Training error is high.
- Test error is high.
- Example: linear model on very nonlinear data.

High variance:

- Model is too complex.
- It learns noise in the training data.
- Training error is low.
- Test error is much higher.
- Example: very deep decision tree with no pruning.

Goal:

```
low enough bias + low enough variance = good generalization
```

Hyperparameter examples:

- Increasing decision tree depth reduces bias but can increase variance.

- Increasing KNN k reduces variance but can increase bias.
- Increasing neural network complexity reduces bias but can increase variance.
- Adding dropout/regularization reduces variance but can increase bias if too strong.

16.5 Data Leakage Theory

Data leakage means the model learns information it would not have in the real world.

Common leakage examples:

- Scaling before train/test split.
- Applying SMOTE before train/test split.
- Fitting PCA on all data before splitting.
- Using test set repeatedly to tune hyperparameters.
- Including future information in features.

Why it is dangerous:

- It creates artificially high test scores.
- The model may fail in real deployment.
- It gives a false sense of model quality.

Correct theory:

```
Training data teaches preprocessing.
Test data only receives already-learned preprocessing.
```

16.6 Regression Theory

Regression predicts continuous numeric targets.

Examples:

- Salary.
- House price.
- Student marks.
- Fuel efficiency.
- Disease progression.

The model learns a function:

```
y_hat = f(x)
```

Regression errors are differences between actual and predicted values:

```
error = y - y_hat
```

Important theoretical difference between metrics:

- MAE treats each error linearly.
- MSE squares errors, so large errors matter much more.
- RMSE returns MSE to the original unit.
- R^2 measures explained variation, not direct error size.

When comparing regression models:

- Lower MAE/MSE/RMSE is better.
- Higher R^2 is better.
- Always compare train and test scores to detect overfitting.

16.7 Linear Regression Theory

Linear regression assumes the target can be approximated by a linear combination of features:

```
y_hat = beta_0 + beta_1*x_1 + beta_2*x_2 + ... + beta_n*x_n
```

Theory:

- `beta_0` is the intercept.
- `beta_i` values are coefficients/slopes.
- Coefficients measure how predictions change when a feature changes.
- The best-fit line minimizes error, commonly squared error.

Simple linear regression has one input feature.

Multiple linear regression has multiple input features.

Interpretation:

- Positive coefficient: as feature increases, prediction tends to increase.
- Negative coefficient: as feature increases, prediction tends to decrease.
- Large coefficient magnitude means stronger influence, but only if features are comparable in scale.

Limitations:

- Linear relationship assumption.
- Sensitive to outliers.
- Multicollinearity can make coefficients unstable.
- Cannot automatically model interactions unless interaction features are added.

16.8 Logistic Regression Theory

Logistic regression is a classification model based on a linear score plus a sigmoid function.

Linear score:

```
z = beta_0 + beta_1*x_1 + ... + beta_n*x_n
```

Sigmoid:

```
p = 1 / (1 + e^(-z))
```

Theory:

- `z` can be any real number.
- Sigmoid converts `z` to probability between 0 and 1.
- The probability is converted to a class using a threshold.
- Default threshold is often 0.5.

Decision boundary:

- In feature space, logistic regression creates a linear boundary.
- The sigmoid curve maps distance from boundary into probability.

Cost function:

- Logistic regression uses cross entropy/log loss.
- Log loss punishes confident wrong predictions strongly.

Gradient descent:

- Optimization updates coefficients to reduce loss.
- Feature scaling helps gradient descent converge more smoothly.

Binary, multinomial, and ordinal:

- Binary logistic regression: two classes.
- Multinomial logistic regression: three or more unordered classes.
- Ordinal logistic regression: three or more ordered classes.

16.9 SMOTE and Class Imbalance Theory

Class imbalance means one class has many more examples than another.

Example from Dry Bean:

- Some bean types have thousands of samples.

- Bombay has far fewer samples.

Problem:

- A model can get high accuracy by favoring majority classes.
- Minority class recall can be poor.

SMOTE:

- Synthetic Minority Over-sampling Technique.
- Creates synthetic examples for minority classes.
- It does not simply duplicate rows; it creates new points between existing minority examples.

Correct use:

- Split first.
- Apply SMOTE only to training data.
- Evaluate on original untouched test data.

Why:

- If synthetic points leak into the test set, evaluation becomes too optimistic.

16.10 KNN Theory

KNN is based on similarity.

Core assumption:

```
points that are close in feature space should have similar labels or target values
```

For classification:

- Nearest neighbors vote.

For regression:

- Nearest neighbors' target values are averaged.

Why scaling matters:

If one feature ranges from 0 to 100000 and another ranges from 0 to 1, the large-scale feature dominates distance.

Distance examples:

- Euclidean distance: straight-line distance.
- Manhattan distance: grid-like absolute distance.

Voronoi idea:

- In 1-NN, each training point owns a region of the feature space.
- New points are classified according to the closest training point.
- These regions form a Voronoi diagram.

k theory:

- k=1: very flexible, high variance.
- Medium odd k: often stable for classification.
- Very large k: high bias, too smooth.

16.11 Decision Tree Theory

Decision trees ask a sequence of questions.

At each node:

```
Is feature <= threshold?
```

The model chooses splits that make child nodes purer.

Classification tree purity:

- Gini impurity.

- Entropy/information gain.

Regression tree purity:

- MSE or MAE reduction.

Why trees do not need scaling:

- A split like `feature <= 5.2` works the same whether another feature has larger units.
- Trees do not compute distances.

Why trees overfit:

- If allowed to grow too deep, a tree can isolate individual training samples.
- This creates very specific rules that do not generalize.

Feature importance:

- A feature is important if it reduces impurity/error across splits.
- In Iris, petal features are very important.
- In email spam, word-count features like `enron`, `ali`, or `http` can become important depending on the trained tree.

16.12 SVM Theory

SVM tries to find a boundary that separates classes with maximum margin.

Core ideas:

- Hyperplane: boundary.
- Margin: gap between boundary and closest data points.
- Support vectors: closest points that define the margin.

The theory of margin:

- A larger margin means the classifier is less sensitive to small changes.
- A small margin may fit training data but generalize poorly.

Hard margin:

- No classification errors allowed.
- Works only when data is perfectly separable.
- Sensitive to outliers.

Soft margin:

- Allows some errors.
- Better for real noisy data.
- Controlled by `C`.

Kernel theory:

- A kernel computes similarity in a transformed feature space.
- It allows nonlinear boundaries without explicitly constructing all transformed features.

Kernel types:

- Linear: straight boundary.
- Polynomial: curved boundary through polynomial interactions.
- RBF: flexible local similarity boundary.
- Sigmoid: related to neural-network-like activation, less commonly stable.

Why SVM needs scaling:

- Margin and kernels depend on distances/dot products.

16.13 SVR Theory

SVR adapts SVM theory to regression.

Instead of separating classes, SVR fits a function with a tolerance tube.

Key idea:

```
errors inside epsilon tube are ignored
errors outside epsilon tube are penalized
```

The epsilon-insensitive tube gives robustness to small noise.

Slack variables:

- Represent points outside the tube.
- Their distance from the tube is penalized.

Parameter theory:

- Larger `epsilon`: wider tube, smoother model, more ignored error.
- Smaller `epsilon`: stricter fit, more points penalized.
- Larger `C`: stronger penalty for errors, tighter fit.
- Smaller `C`: more tolerance, smoother but possibly underfit.

16.14 Clustering Theory

Clustering tries to discover groups without labels.

Important theory:

- Clustering does not know the "true class" unless labels are later used for comparison.
- A cluster is based on similarity, not necessarily the same as a real-world category.
- Human interpretation is often needed.

Types:

- Partitional clustering: divides data into non-overlapping groups.
- Hierarchical clustering: creates nested group structure.
- Density-based clustering: finds dense regions and noise.

16.15 K-Means Theory

K-means minimizes within-cluster distances to centroids.

Objective:

```
minimize total squared distance from points to assigned centroid
```

Why inertia decreases as `k` increases:

- More clusters means points can be closer to a center.
- But too many clusters may over-segment data.

Elbow theory:

- Find the point where adding more clusters gives diminishing returns.

Limitations:

- Must choose `k`.
- Sensitive to outliers.
- Sensitive to feature scale.
- Sensitive to initialization.
- Works best with compact, roughly spherical clusters.

16.16 K-Medoids Theory

K-medoids is similar to K-means but centers are actual observations.

Why it is robust:

- Outliers affect means strongly.
- Medoids must be real points, so they are less pulled by extreme values.

Tradeoff:

- More robust but slower.

- Often higher inertia than K-means because the exact mean may not be a real data point.

16.17 Hierarchical Clustering Theory

Agglomerative clustering:

```
start with each point as its own cluster -> repeatedly merge closest clusters
```

Linkage defines "closest":

- Single linkage: minimum distance between points in clusters.
- Complete linkage: maximum distance.
- Average linkage: average distance.
- Ward linkage: minimizes increase in variance.

Dendrogram theory:

- Merge height shows dissimilarity.
- Low merge height means clusters are similar.
- High merge height means clusters are different.
- A horizontal cut produces a chosen number of clusters.

16.18 DBSCAN Theory

DBSCAN is based on density, not centroids.

Core point:

- Has at least `min_samples` points within `eps` distance.

Border point:

- Not dense enough to be core.
- Still reachable from a core point.

Noise point:

- Not core and not reachable from a core point.

Why DBSCAN can find arbitrary shapes:

- Clusters grow through density-connected points, not through distance to a center.

Why parameter choice matters:

- `eps` defines neighborhood size.
- `min_samples` defines how dense a region must be.
- Too large `eps` merges clusters.
- Too small `eps` creates many noise points.

16.19 Dimensionality Reduction Theory

High-dimensional data can be difficult because:

- More features increase computational cost.
- Models can overfit.
- Distance becomes less meaningful.
- Visualization is hard.
- Features may be redundant or highly correlated.

Dimensionality reduction solves this by:

- Removing features.
- Combining features.
- Projecting data to a smaller space.

Tradeoff:

- Fewer dimensions and less noise.
- Possible loss of information and interpretability.

16.20 PCA Theory

PCA finds new axes called principal components.

Theory:

- PC1 captures the most variance.
- PC2 captures the next most variance while orthogonal to PC1.
- Components are linear combinations of original features.
- Components are uncorrelated/orthogonal.

Explained variance ratio:

```
variance explained by component / total variance
```

If first two components explain:

```
[0.4427, 0.1897]
```

Then:

- PC1 explains about 44.27%.
- PC2 explains about 18.97%.
- Together they explain about 63.24%.

Why scaling matters:

- PCA maximizes variance.
- Large-scale features can dominate variance even if not more important.

`n_components` choices:

- Integer like `2`: keep exactly 2 components.
- Float like `0.95`: keep enough components to preserve 95% variance.

16.21 LDA Theory

LDA projects data to maximize class separation.

It considers:

- Between-class variance: classes should be far apart.
- Within-class variance: samples in same class should stay close.

Goal:

```
maximize between-class separation / within-class spread
```

The number of components is limited by number of classes:

```
max_components = min(n_classes - 1, n_features)
```

Why:

- With `c` classes, there are at most `c - 1` independent directions that separate class means.

16.22 t-SNE Theory

t-SNE is designed for visualization.

High-dimensional space:

- Computes pairwise similarities.
- Nearby points get high similarity.
- Far points get low similarity.

Low-dimensional space:

- Places points so neighborhood relationships are preserved.

Perplexity:

- Controls effective number of neighbors.
- Too low can fragment clusters.
- Too high can blur local structure.

Important warning:

- t-SNE plot clusters are useful visually, but distances between far clusters are not always meaningful.

16.23 UMAP Theory

UMAP assumes high-dimensional data lies on a manifold.

Theory:

- Builds a nearest-neighbor graph.
- Represents local structure using fuzzy topology.
- Optimizes low-dimensional points to preserve that structure.

`n_neighbors`:

- Small values emphasize local details.
- Large values preserve broader structure.

`min_dist`:

- Low values allow tight clusters.
- High values force points to spread out.

UMAP is often faster than t-SNE and scales better to large datasets.

16.24 Semi-Supervised Learning Theory

Semi-supervised learning relies on assumptions:

Smoothness assumption:

- Points close together should have similar labels.

Cluster assumption:

- Points in the same cluster likely share a label.

Manifold assumption:

- Data lies on a lower-dimensional structure, and labels vary smoothly along it.

Self-training:

- Uses model confidence to create pseudo-labels.
- Risk: wrong pseudo-labels can reinforce errors.

Label propagation:

- Uses graph similarity.
- Risk: noisy graph connections can spread wrong labels.

16.25 Neural Network Theory

Neural networks learn by adjusting weights and biases to reduce loss.

Forward pass:

```
input -> weighted sums -> activations -> prediction
```

Loss:

```
prediction compared with true label
```

Backpropagation:

- Computes how much each weight contributed to error.
- Gradients are used to update weights.

Optimizer:

- Uses gradients to change weights.
- Adam is popular because it adapts learning rates.

Why activation functions matter:

- Without nonlinear activation, stacked layers collapse into a linear model.
- ReLU, sigmoid, tanh, and softmax allow different kinds of nonlinear behavior.

16.26 CNN Theory

CNNs work well for images because of:

Local receptive fields:

- Filters look at small local image regions.

Parameter sharing:

- The same filter slides across the image.
- This greatly reduces parameters compared with fully connected image models.

Spatial hierarchy:

- Early layers learn edges.
- Middle layers learn shapes/textures.
- Later layers learn object parts/classes.

Pooling:

- Reduces spatial size.
- Adds some translation robustness.
- Reduces computation.

Output size theory for convolution:

```
output_size = floor((input_size - kernel_size + 2*padding) / stride) + 1
```

For valid padding:

```
padding = 0
```

For same padding and stride 1:

```
output_size = input_size
```

16.27 Learning Behavior Theory

Training curves are diagnostic tools.

Overfitting theory:

- Model capacity is too high relative to data.
- It memorizes noise.
- Regularization, dropout, augmentation, and early stopping help.

Underfitting theory:

- Model capacity is too low or training is insufficient.
- It cannot learn the underlying pattern.
- More complexity or longer training can help.

Dropout theory:

- Randomly removes neurons during training.

- Forces distributed representations.
- Acts like averaging many subnetworks.

Early stopping theory:

- Prevents later epochs from overfitting.
- Keeps model near best validation performance.

Data augmentation theory:

- Creates realistic variations.
- Teaches invariance to rotation, zoom, flip, etc.

Regularization theory:

- Penalizes overly large or unnecessary weights.
- Encourages simpler functions.

16.28 Transfer Learning Theory

Deep CNNs learn reusable feature hierarchies.

For ImageNet pretrained models:

- Early layers learn generic edges and textures.
- Middle layers learn shapes and object parts.
- Later layers learn dataset-specific class combinations.

Transfer learning uses this hierarchy.

Two main modes:

Feature extraction:

- Freeze base model.
- Train only new classifier head.
- Best when dataset is small.

Fine-tuning:

- Unfreeze some top layers.
- Train with low learning rate.
- Best when you have more data or task differs from original.

`include_top=False` theory:

- Removes original ImageNet classifier.
- Keeps convolutional feature extractor.
- Allows custom output layer for your classes.

Why VGG is heavy:

- Large fully connected layers contain most parameters.
- Good feature extractor but expensive.

Why MobileNet is lightweight:

- Designed for efficient inference.
- Much smaller and faster.

17. Final Summary Tables

17.1 Deep Comparison of Regressors

Regressor	Core idea	Needs scaling?	Handles nonlinearity?	Main hyperparameters	Advantages	Weaknesses	Most us cases
Linear Regression	Fits best straight line/hyperplane by learning coefficients and intercept	Not strictly for plain linear regression	Only linear unless features are engineered	Usually few; fit intercept, regularization if added	Simple, fast, interpretable, strong baseline	Sensitive to outliers, poor for complex nonlinear patterns	Salary predictio simple forecasti baseline regressic

KNN Regressor	Predicts average/weighted average of nearest neighbors	Yes	Yes, local nonlinear patterns	`n_neighbors`, distance metric, weights	Simple, non-parametric, can model local behavior	Slow prediction, sensitive to scale/noise/irrelevant features, weak in high dimensions	Small/me datasets where si inputs ha similar outputs
Decision Tree Regressor	Recursively splits features to reduce prediction error; leaf predicts mean target	No	Yes	`max_depth`, `min_samples_split`, `min_samples_leaf`, `criterion`	Interpretable, scale-free, captures nonlinear rules	Overfits easily, step-like predictions, unstable	Tabular nonlinea regressic explainab rules
Linear SVR	Fits a line with an epsilon-insensitive error tube	Yes	Linear only	`C`, `epsilon`, `kernel="linear"`	Margin-based, robust to small errors inside tube	Sensitive to scaling and hyperparameters	Smaller regressic tasks wit mostly li pattern
RBF SVR	Fits nonlinear function using RBF kernel and epsilon tube	Yes	Yes, flexible	`C`, `epsilon`, *`gamma`*, `kernel="rbf"`	Strong nonlinear modeling, can perform very well	Slow on large data, can overfit, needs tuning	Nonlinea regressic with medium/ data
Polynomial SVR	Uses polynomial kernel to model structured feature interactions	Yes	Yes	`C`, `epsilon`, `degree`, `coef0`	Captures polynomial relationships	Expensive and sensitive to degree	Data wit polynom like interactic
Neural Network Regressor	Learns nonlinear function through layers of weights, biases, activations	Yes	Yes, very flexible	Layers, neurons, activation, optimizer, learning rate, epochs, batch size	Powerful for complex data	Needs data, tuning, compute; less interpretable	Images, signals, complex nonlinea regressic

17.2 Classifier Comparison

Classifier	Output type	Needs scaling?	Decision boundary	Strengths	Weaknesses	Best use cases
Logistic Regression	Binary class/probability	Yes, recommended	Linear boundary after sigmoid	Simple, interpretable, probabilistic	Weak for nonlinear patterns	Binary classification baseline, Titanic-style tabular data
Multinomial Logistic Regression	3+ unordered classes	Yes, recommended	Linear class boundaries	Good multiclass baseline, probabilistic	Needs scaling and balanced data handling	Dry Bean-style multiclass tabular classification
KNN Classifier	Class by majority vote	Yes	Irregular local boundary	Simple, non-parametric, no training cost	Slow prediction, scale-sensitive	Small/medium data with local clusters
Decision Tree Classifier	Class from leaf node	No	Axis-aligned rule splits	Interpretable, handles nonlinear rules, feature importance	Overfits if deep	Explainable tabular classification, spam word counts
Linear SVM	Class from side of hyperplane	Yes	Linear maximum-margin boundary	Strong in high-dimensional data	Needs scaling, tuning	Text/spam, linearly separable data
RBF SVM	Class using nonlinear kernel	Yes	Smooth nonlinear boundary	Powerful for complex tabular patterns	Can overfit, slower	Medium-sized nonlinear tabular data

CNN	Class probabilities	Pixel normalization required	Learned spatial feature hierarchy	Best for images, learns features automatically	Needs compute/data, less interpretable	MNIST, image classification
Self-Training Classifier	Semi-supervised labels	Usually depends on base model	Depends on base model	Uses unlabeled data, reduces labeling cost	Pseudo-label errors can reinforce mistakes	Small labeled + large unlabeled dataset
Label Propagation	Semi-supervised labels	Yes, usually	Graph similarity structure	Spreads labels through data geometry	Sensitive to noise/similarity graph	When similar points likely share class

17.3 Scaling Methods

Scaler	Formula idea	Best for	Sensitive to outliers?	Typical models
StandardScaler	`(x - mean) / std`	Roughly symmetric data, optimization methods	Yes	SVM, KNN, logistic regression, PCA, neural networks
MinMaxScaler	`(x - min) / (max - min)`	Fixed 0-1 range	Yes	Neural networks, image pixels, bounded features
RobustScaler	`(x - median) / IQR`	Data with outliers	Less sensitive	KNN/SVM/logistic when outliers exist
Pixel normalization	`pixel / 255`	Images with 0-255 pixel values	N/A	Dense image models, CNNs

17.4 Which Algorithms Need Scaling?

Algorithm	Scaling needed?	Why
Linear Regression	Usually no	Coefficients adjust to scale, though scaling helps regularized versions
Logistic Regression	Yes, recommended	Optimization and weights behave better
Multinomial Logistic Regression	Yes	Prevents large-scale features from dominating
KNN	Yes	Distance calculation depends directly on feature scale
SVM/SVR	Yes	Margin and kernels depend on distances/dot products
Decision Tree	No	Splits by thresholds, not distances
K-Means	Yes	Uses distances to centroids
K-Medoids	Yes	Uses distances to medoids
Hierarchical Clustering	Yes	Uses distances between points/clusters
DBSCAN	Yes	Uses epsilon-neighborhood distance
PCA	Yes	Variance is scale-sensitive
LDA	Recommended	Class separation can be dominated by large-scale features
t-SNE	Recommended	Neighbor relationships depend on distances
UMAP	Recommended	Neighbor graph depends on distances
Neural Networks	Yes	Training converges better
CNNs	Yes	Normalize image pixels

17.5 Dimensionality Reduction Comparison

Method	Supervised?	Linear/nonlinear	Main goal	Important parameters	Scaling?	Best use
PCA	No	Linear	Preserve variance	`n_components`	Yes	Reduce correlated features, visualization, preprocessing

LDA	Yes	Linear	Maximize class separation	`n_components <= min(classes-1, features)`	Recommended	Classification-focused projection
t-SNE	No	Nonlinear	Preserve local neighborhoods for visualization	`n_components`, `perplexity`, `random_state`	Recommended	Visualize high-dimensional data or CNN features
UMAP	No, usually	Nonlinear	Preserve local and some global manifold structure	`n_neighbors`, `min_dist`, `n_components`	Recommended	Faster large-scale visualization/embedding
Autoencoder	Usually no labels	Nonlinear	Learn compressed representation through neural network	architecture, latent size, loss	Yes	Complex nonlinear compression

17.6 Clustering Comparison

Method	Need number of clusters?	Center type	Handles outliers?	Cluster shape	Main evaluation/help tool	Best use
K-Means	Yes, k	Centroid mean	Poorly	Spherical/compact	Inertia, elbow method, silhouette	Large well-separated numeric data
K-Medoids	Yes, k	Actual data point	Better than K-means	Compact	Inertia, elbow method, silhouette	Smaller noisy datasets
Hierarchical	Can choose from dendrogram	No fixed center	Depends on linkage	Nested clusters	Dendrogram, silhouette	Understanding merge structure
DBSCAN	No	Density regions	Yes, marks noise	Arbitrary shapes	Noise count, cluster count, parameter sweep	Density clusters and outlier detection

17.7 SVM and SVR Hyperparameters

Parameter	Used in	Meaning	If increased
`C`	SVM/SVR	Penalty for errors	Less tolerance, tighter fit, possible overfitting
`epsilon`	SVR	Width of no-penalty tube	More errors ignored, smoother fit
`kernel`	SVM/SVR	Boundary/function type	Changes linear vs nonlinear behavior
`gamma`	RBF SVM/SVR	Influence range of points	Higher gamma creates more local/complex boundary
`degree`	Polynomial SVM/SVR	Polynomial complexity	Higher degree creates more complex curve

17.8 Decision Tree Hyperparameters

Parameter	Meaning	Overfitting effect
`max_depth`	Maximum tree depth	Higher depth can overfit
`min_samples_split`	Minimum samples required to split node	Higher value reduces overfitting
`min_samples_leaf`	Minimum samples required in a leaf	Higher value smooths tree
`criterion`	Split quality measure	Classification: `gini`, `entropy`; regression: `squared_error`, `absolute_error`
`max_features`	Features considered per split	Lower value can reduce overfitting

17.9 Neural Network and CNN Layer Table

Layer/Concept	What it does	Common use
Dense	Fully connected layer	Tabular data, classifier head
Flatten	Converts multidimensional tensor to 1D vector	Before dense layers
Conv2D	Learns image filters/features	CNN feature extraction
ReLU	Adds nonlinearity, keeps positive values	Hidden layers, CNNs
MaxPool2D	Downsamples feature maps	Reduces spatial size
Dropout	Randomly disables neurons during training	Reduce overfitting
Softmax	Converts outputs to multiclass probabilities	Multiclass output
Sigmoid	Converts output to binary probability	Binary output
Batch size	Samples per weight update	Training configuration
Epoch	One full pass over training data	Training duration
Early stopping	Stops when validation stops improving	Prevent overfitting

17.10 Loss Functions and Output Layers

Task	Output layer	Loss function	Example
Binary classification	`Dense(1, activation="sigmoid")`	`binary_crossentropy`	Survived/not survived
Multiclass classification with integer labels	`Dense(num_classes, activation="softmax")`	`sparse_categorical_crossentropy`	MNIST digits 0-9
Multiclass classification with one-hot labels	`Dense(num_classes, activation="softmax")`	`categorical_crossentropy`	One-hot image labels
Regression	`Dense(1)` with linear activation	`mse`, `mae`	House price, marks

17.11 Evaluation Metrics Table

Metric	Task	Better value	Meaning
Accuracy	Classification	Higher	Fraction of correct predictions
Precision	Classification	Higher	Correct predicted positives out of predicted positives
Recall	Classification	Higher	Correct predicted positives out of actual positives
F1-score	Classification	Higher	Balance of precision and recall
Confusion matrix	Classification	Depends	Counts correct and incorrect predictions by class
Log loss	Classification probabilities	Lower	Probability error; confident wrong predictions punished
MAE	Regression	Lower	Average absolute error
MSE	Regression	Lower	Average squared error
RMSE	Regression	Lower	Square root of MSE, same unit as target
R^2	Regression	Higher	Explained variance
Inertia	Clustering	Lower for fixed k	Compactness of clusters
Silhouette	Clustering	Higher	Separation and cohesion of clusters

17.12 Final Big-Picture Algorithm Selection Table

Problem situation	Good first choices	Why
Predict a numeric value with simple trend	Linear regression	Fast, interpretable baseline
Predict a numeric value with nonlinear tabular patterns	Decision tree regressor, KNN regressor, RBF SVR	Capture nonlinear behavior

Binary tabular classification	Logistic regression, decision tree, SVM	Strong baselines
Multiclass tabular classification	Multinomial logistic regression, decision tree, SVM, KNN	Supports 3+ classes
Image classification from pixels	CNN	Learns visual features automatically
Small image dataset	Transfer learning	Uses pretrained visual features
Unlabeled grouping with known k	K-means or K-medoids	Partitional clustering
Unlabeled grouping with outliers	DBSCAN or K-medoids	Handles noise better
Need cluster hierarchy	Hierarchical clustering	Dendrogram shows nested structure
Too many correlated features	PCA	Combines variance into components
Need class-separating projection	LDA	Uses labels to separate classes
Need visualize high-dimensional features	t-SNE or UMAP	Reduces to 2D/3D for plots
Few labels, many unlabeled samples	Self-training or label propagation	Semi-supervised learning

17.13 Final Exam Checklist

Question to ask yourself	What to remember
Is the target numeric or categorical?	Numeric = regression; categorical = classification
Are labels available?	Labels = supervised; no labels = unsupervised; some labels = semi-supervised
Does the algorithm use distance or gradient optimization?	Scale the data
Is the model a decision tree?	Scaling usually not needed
Are classes imbalanced?	Use confusion matrix, precision/recall/F1; consider SMOTE on training data only
Is train score much better than test score?	Overfitting
Are both train and test scores poor?	Underfitting
Is this image data?	CNN or transfer learning
Do you need visualization of high-dimensional data?	PCA, t-SNE, or UMAP
Do you need explainability?	Linear/logistic regression or decision tree
Do you need flexible nonlinear boundaries?	RBF SVM, KNN, decision tree, neural network
Do you know the number of clusters?	If yes, K-means/K-medoids; if no, DBSCAN or dendrogram/silhouette